Graph Mining

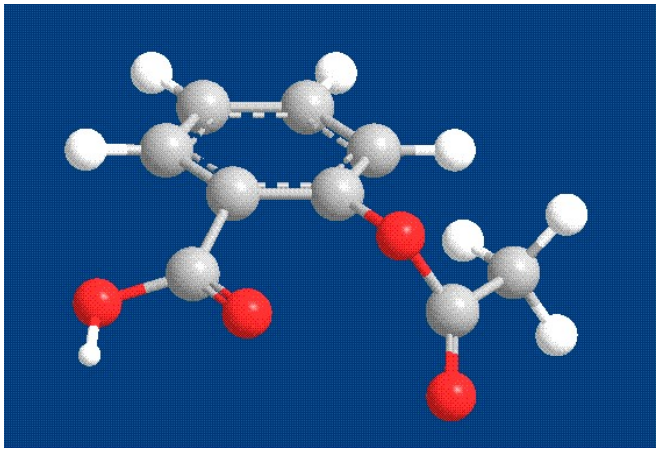
Graph Mining

- Why Graph Mining?
- Methods for Mining Frequent Subgraphs
- Graph Classification
- Graph Clustering

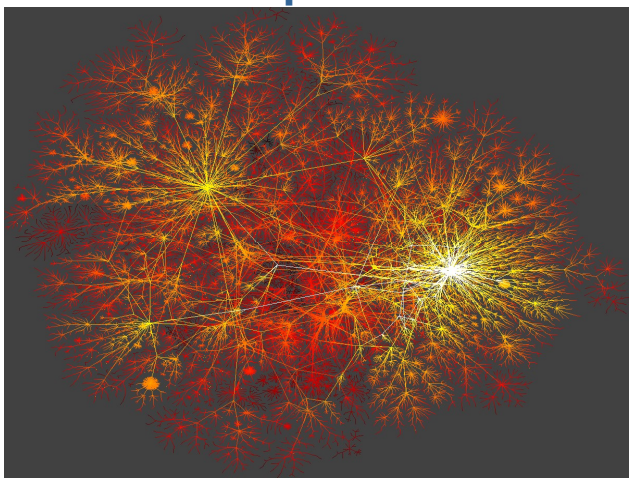
Why Graph Mining?

- Graphs are ubiquitous
  - Chemical compounds (Cheminformatics)
  - Protein structures, biological pathways/networks (Bioinformactics)
  - Program control flow, traffic flow, and workflow analysis
  - XML databases, Web, and social network analysis
- Graph is a general model
  - Trees, lattices, sequences, and items are degenerated graphs
- Diversity of graphs
  - Directed vs. undirected, labeled vs. unlabeled (edges & vertices), weighted, with angles & geometry (topological vs. 2-D/3-D)
- Complexity of algorithms: many problems are of high complexity

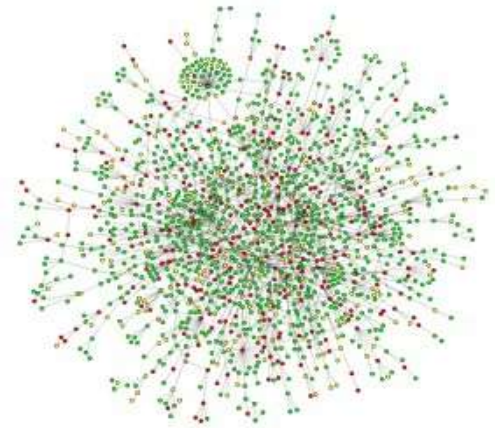
Graph Everywhere



Aspirin

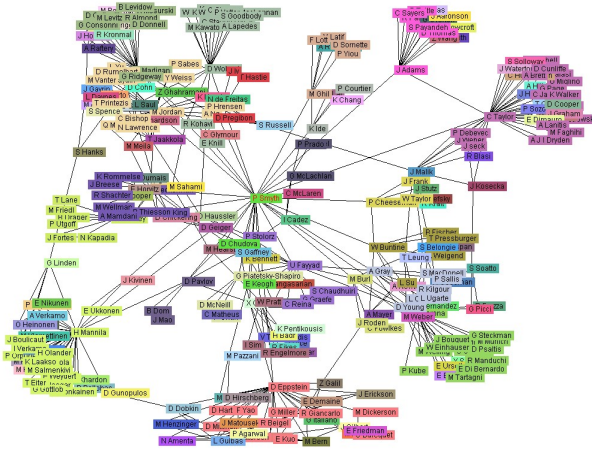


Internet



from H. Jeong et al Nature 411, 41 (2001)

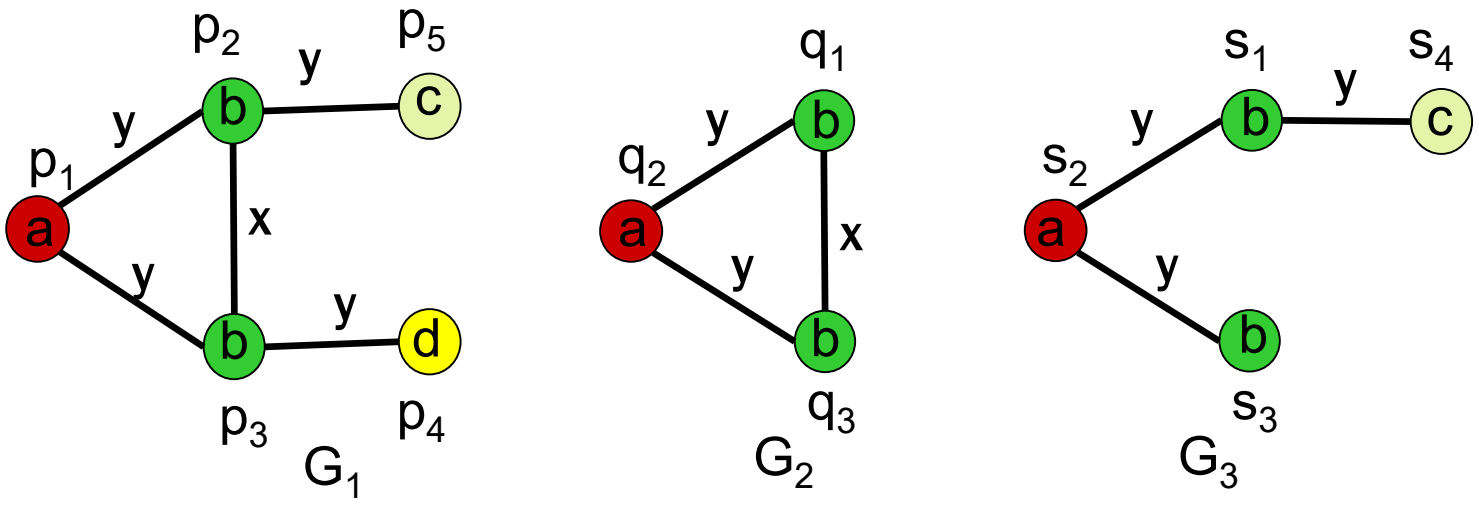
Yeast protein interaction network



Co-author network

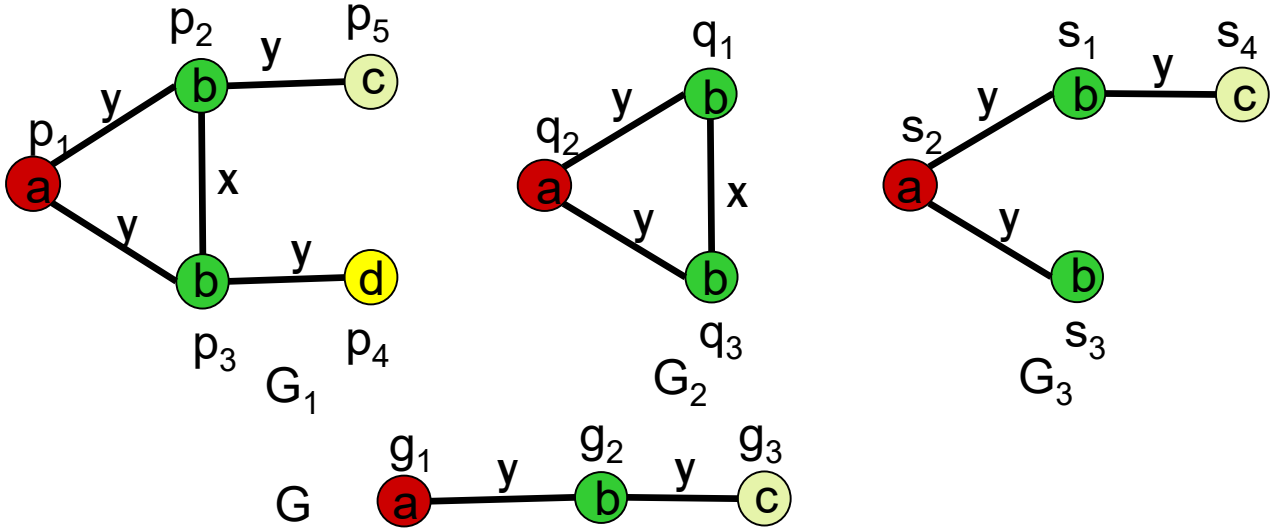
Labeled Graphs

A ***labeled graph*** is a graph where each node and each edge has a label.



Pattern Matching

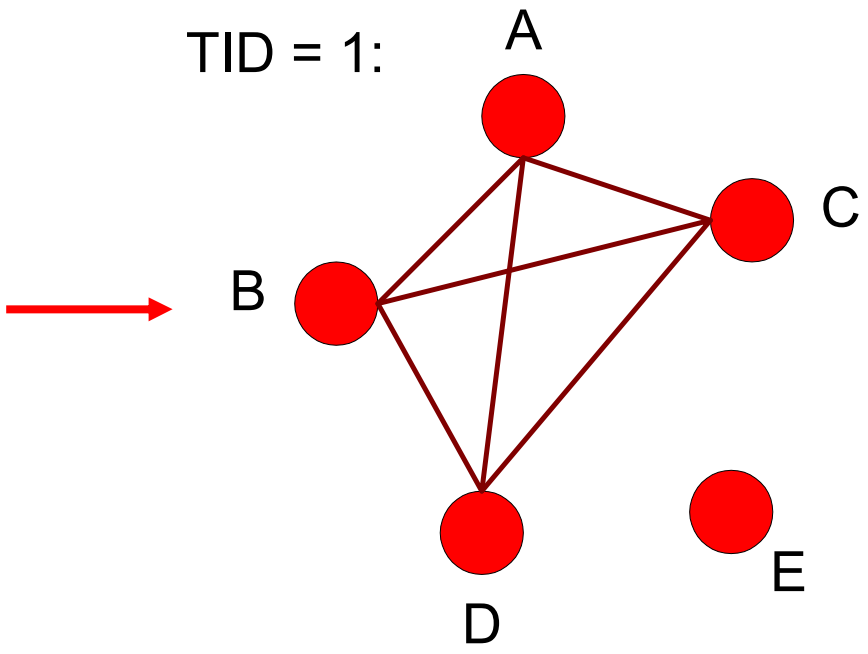
- A graph G is ***subgraph isomorphic*** to a graph G', denoted by $G \subseteq G'$, if
  - there exists a 1-1 mapping from nodes in G to G' such that node labels, edges, and edge labels are preserved with the mapping.
- A ***pattern*** is a graph. Pattern G ***matches*** G' if $G \subseteq G'$
  - G ***occurs*** in G' if $G \subseteq G'$.
  - With a label set, a ***graph space*** is a collection of graphs whose labels are from the set.



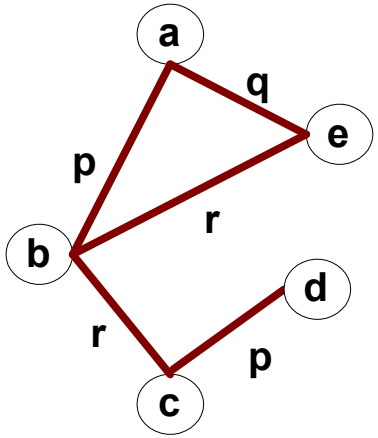
Representing Transactions as Graphs

- Each transaction is a clique of items

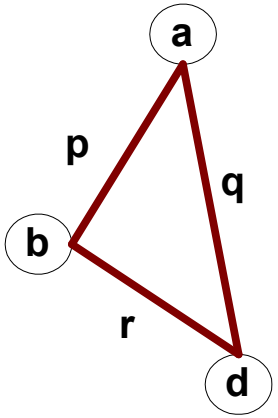
Transaction Id	Items
1	{A,B,C,D}
2	{A,B,E}
3	{B,C}
4	{A,B,D,E}
5	{B,C,D}



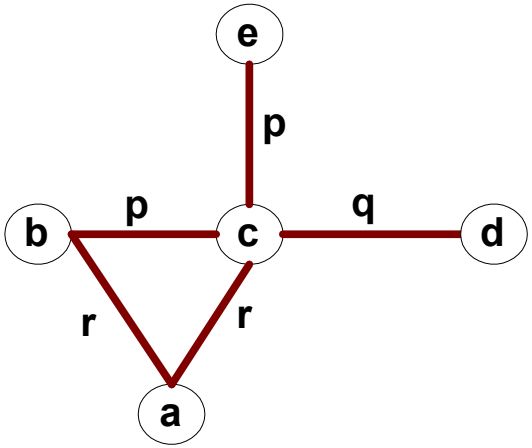
Representing Graphs as Transactions



G1



G2

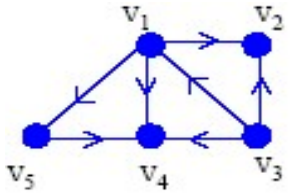


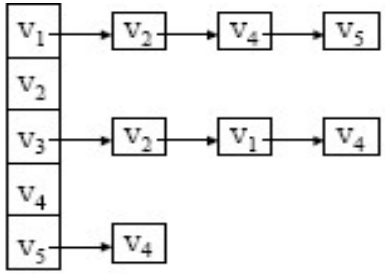
G3

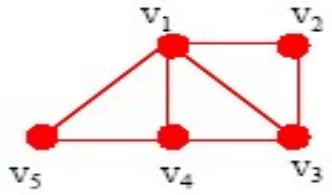
	(a,b,p)	(a,b,q)	(a,b,r)	(b,c,p)	(b,c,q)	(b,c,r)	…	(d,e,r)
G1	1	0	0	0	0	1	…	0
G2	1	0	0	0	0	0	…	0
G3	0	0	1	1	0	0	…	0
G3	…	…	…	…	…	…	…	…

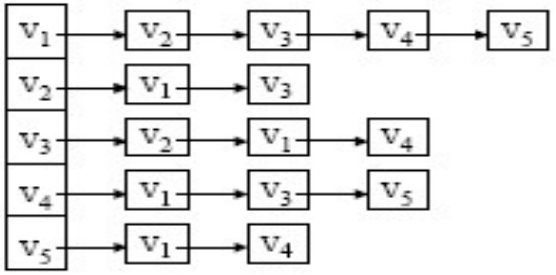
Graph representation

- adjacency list

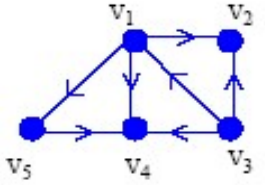




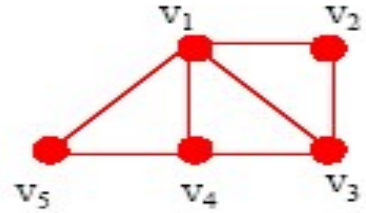




- adjacency matrix

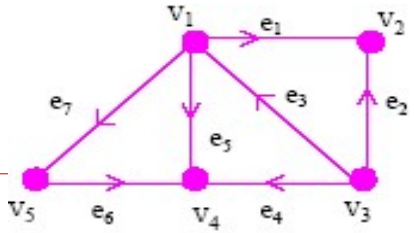


	v_1	v_2	v_3	v_4	v_5
v_1	0	1	0	1	0
v_2	0	0	0	0	0
v_3	1	1	0	1	0
v_4	0	0	0	0	0
v_5	0	0	0	1	0



	v_1	v_2	v_3	v_4	v_5
v_1	0	1	1	1	1
v_2	1	0	1	0	0
v_3	1	1	0	1	0
v_4	1	0	1	0	1
v_5	1	0	0	1	0

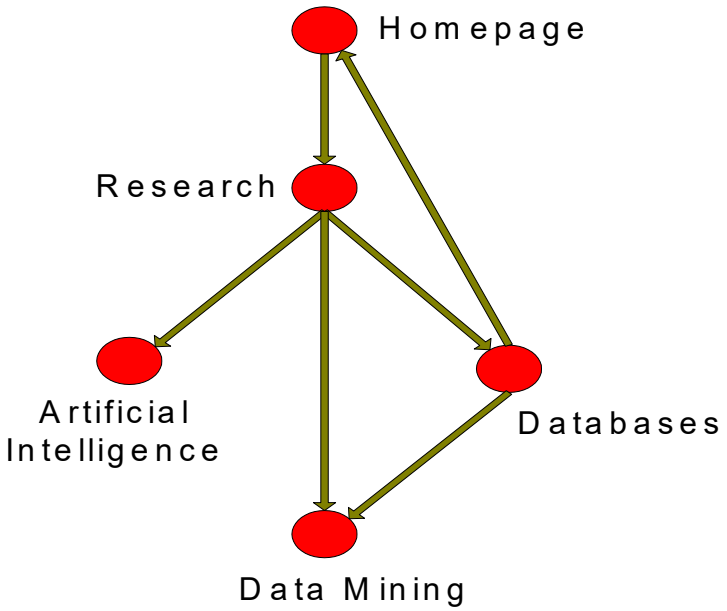
- incidence matrix

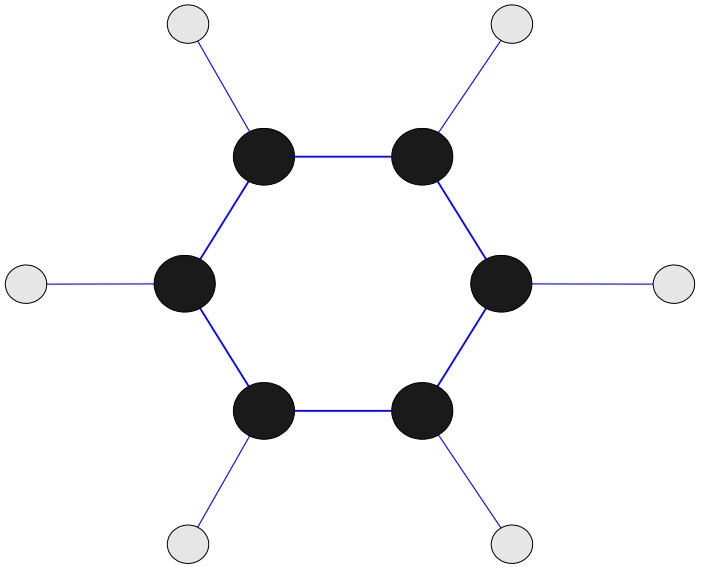


	e_1	e_2	e_3	e_4	e_5	e_6	e_7
v_1	-1	0	1	0	-1	0	-1
v_2	1	1	0	0	0	0	0
v_3	0	-1	-1	-1	0	0	0
v_4	0	0	0	1	1	1	0
v_5	0	0	0	0	0	-1	1

Frequent Subgraph Mining

- Extends association analysis to finding frequent subgraphs
- Useful for Web Mining, computational chemistry, bioinformatics, spatial data sets, etc



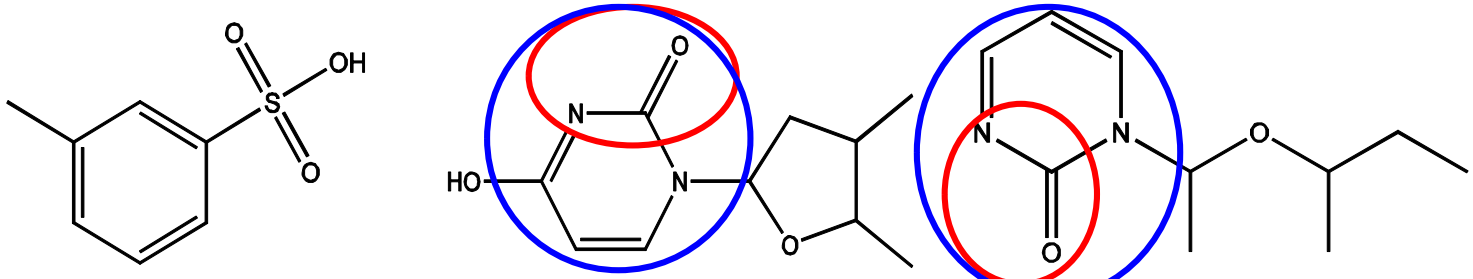


Frequent Subgraph Mining

- *Frequent* subgraphs
  - A (sub)graph is ***frequent*** if its *support* (occurrence frequency) in a given dataset is no less than a *minimum support* threshold
- Applications of graph pattern mining
  - Mining biochemical structures
  - Program control flow analysis
  - Mining XML structures or Web communities
  - Building blocks for graph classification, clustering, compression, comparison, and correlation analysis

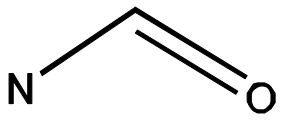
Example: Frequent Subgraphs

Graph Dataset

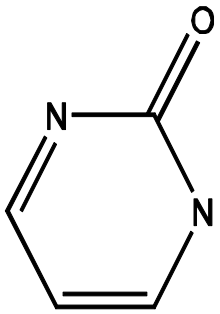


(A) (B) (C)

Frequent Patterns (min support is 2)

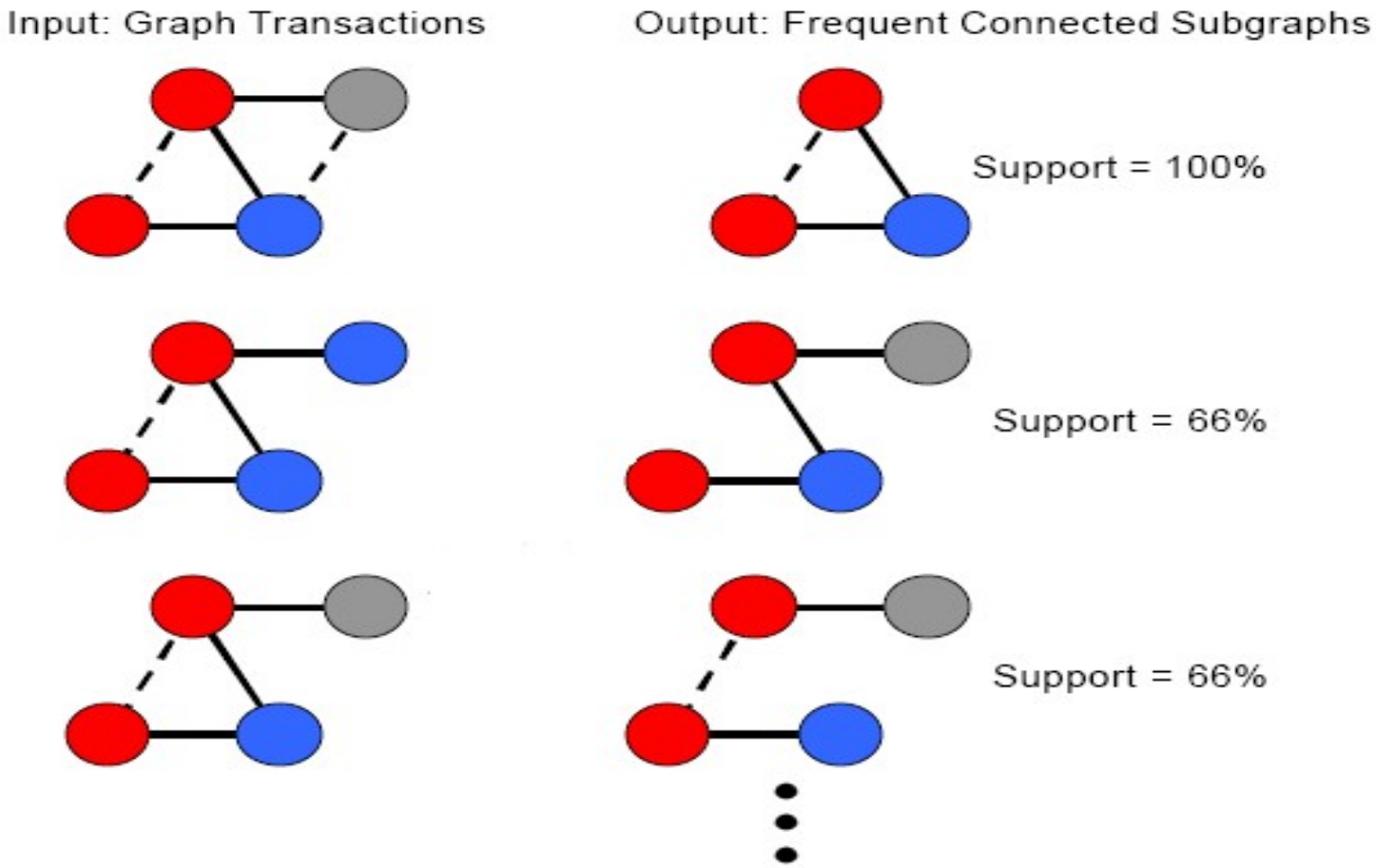


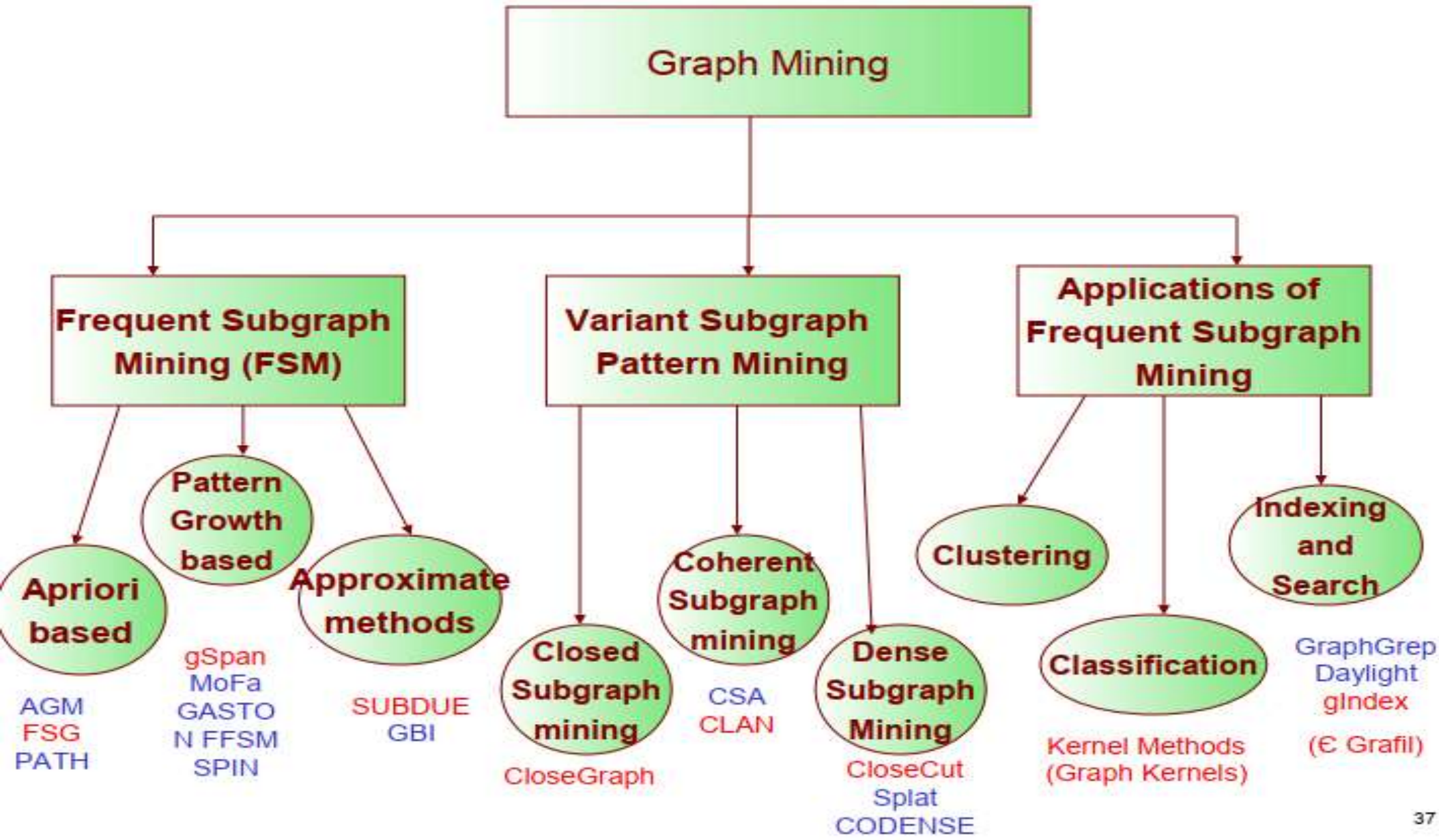
(1)



(2)

Finding frequent subgraphs




Graph Mining
Frequent Subgraph Mining (FSM)
Variant Subgraph Pattern Mining
Applications of Frequent Subgraph Mining
Apriori based
Pattern Growth based
Approximate methods
Closed Subgraph mining
Coherent Subgraph mining
Dense Subgraph Mining
Clustering
Classification
Indexing and Search
AGM
FSG
PATH
gSpan
MoFa
GASTO
N FFSM
SPIN
SUBDUE
GBI
CloseGraph
CSA
CLAN
CloseCut
Splat
CODENSE
Kernel Methods
(Graph Kernels)
GraphGrep
Daylight
gIndex
(Є Grafil)

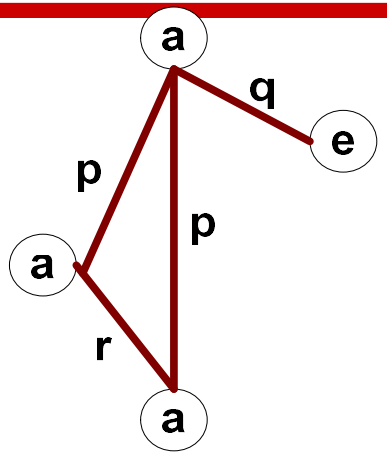
Challenges

- Node may contain duplicate labels
- Support and confidence
  - How to define them?
- Additional constraints imposed by pattern structure
  - Support and confidence are not the only constraints
  - Assumption: frequent subgraphs must be connected
- Apriori-like approach:
  - Use frequent k-subgraphs to generate frequent (k+1) subgraphs
    - What is k?

Challenges…

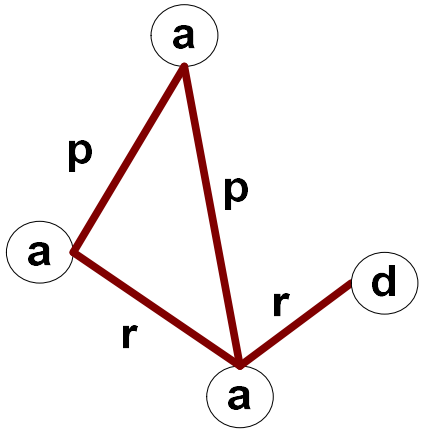
- Support:
  - number of graphs that contain a particular subgraph
- Apriori principle still holds
- Level-wise (Apriori-like) approach:
  - Vertex growing:
    - k is the number of vertices
  - Edge growing:
    - k is the number of edges

Vertex Growing

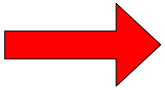


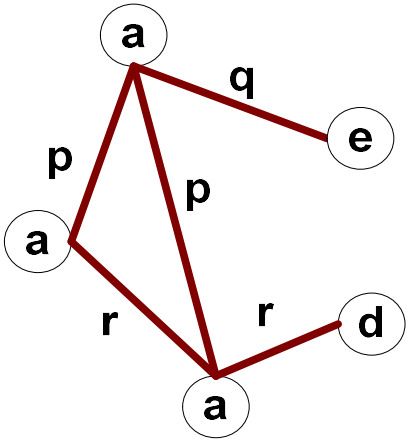
G1

+



G2





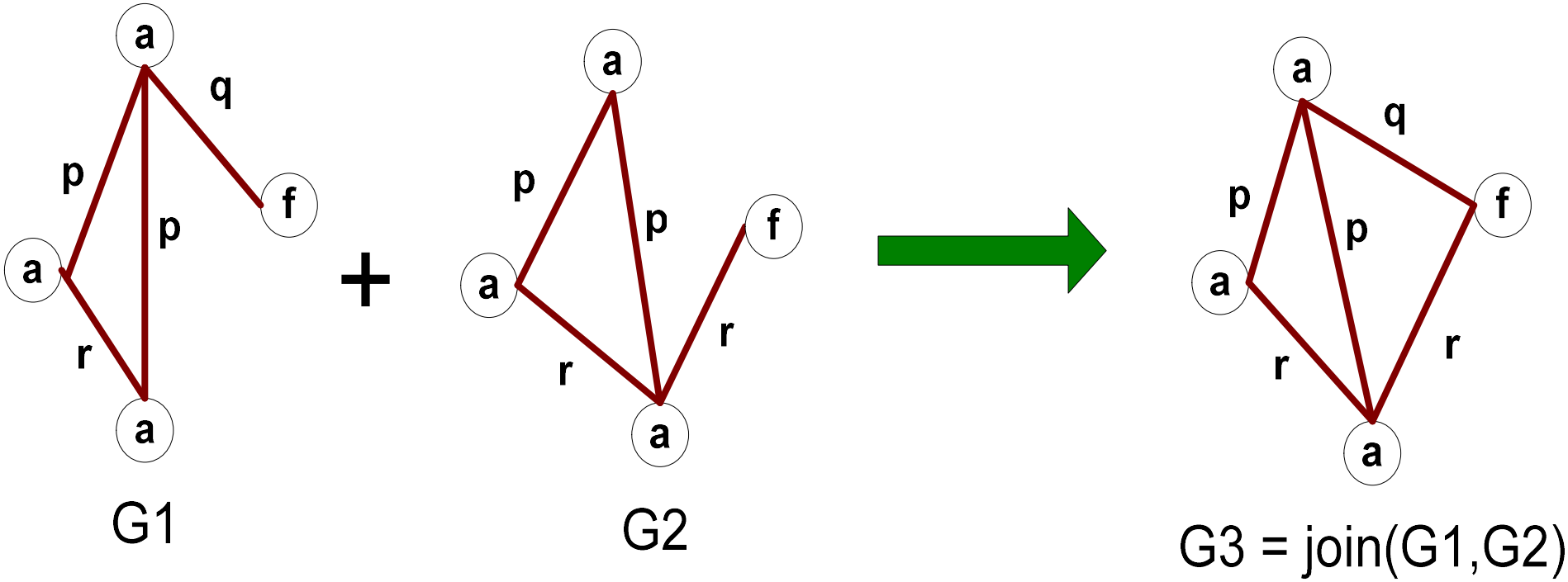
G3 = join(G1,G2)

$$M_{G1} = \begin{pmatrix} 0 & p & p & q \\ p & 0 & r & 0 \\ p & r & 0 & 0 \\ q & 0 & 0 & 0 \end{pmatrix}$$

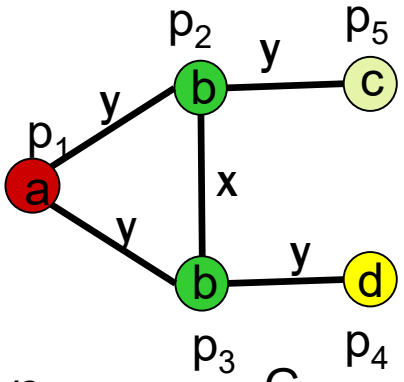
$$M_{G2} = \begin{pmatrix} 0 & p & p & 0 \\ p & 0 & r & 0 \\ p & r & 0 & r \\ 0 & 0 & r & 0 \end{pmatrix}$$

$$M_{G3} = \begin{pmatrix} 0 & p & p & q & 0 \\ p & 0 & r & 0 & 0 \\ p & r & 0 & 0 & r \\ q & 0 & 0 & 0 & ? \\ 0 & 0 & r & ? & 0 \end{pmatrix}$$

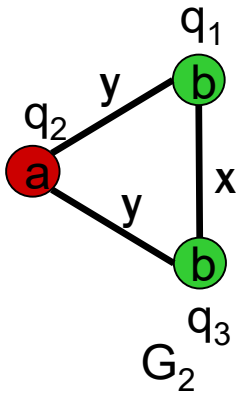
Edge Growing

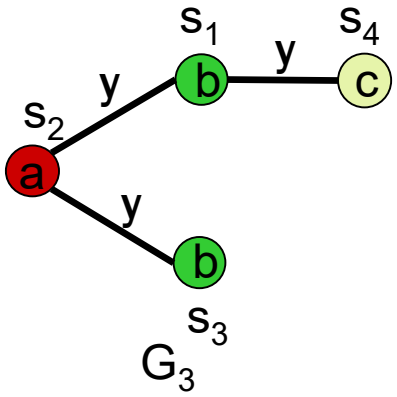


Examples



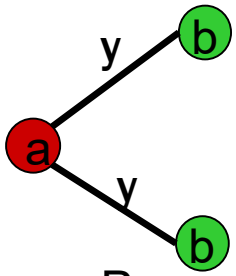
σ = 2/3



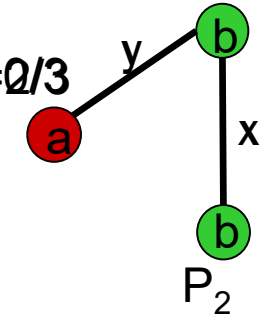


The induced subgraph isomorphism penalizes any unmatched edges

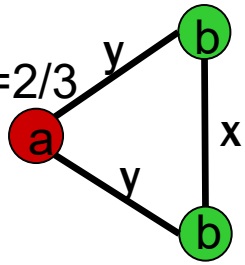
f = 3/3



f=2/3



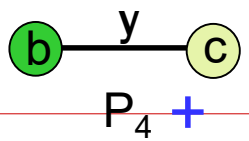
f=2/3



+

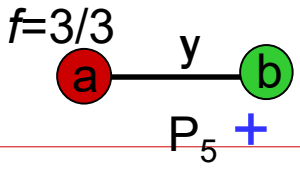
+: induced frequent subgraphs

f=2/3

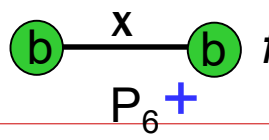


+

f=3/3



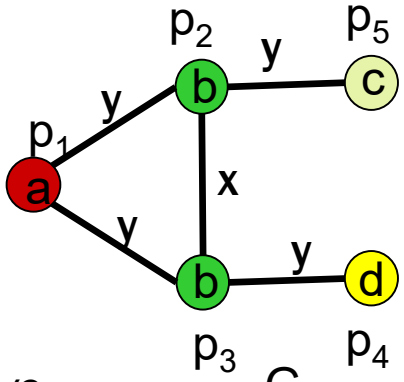
+



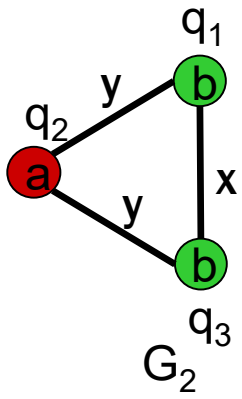
f=2/3

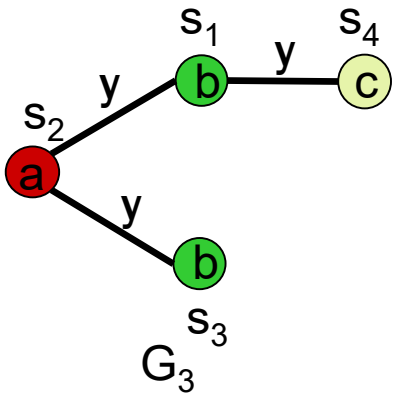
+

Examples



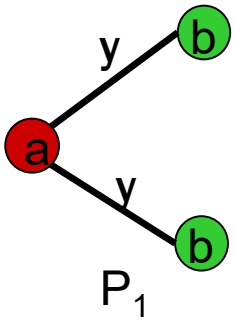
σ = 2/3

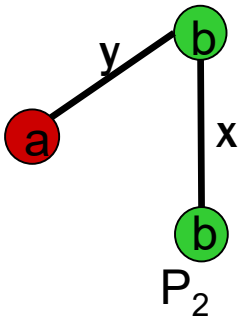


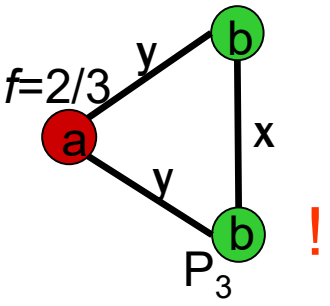


Maximal frequent subgraph are ones that none of their supergraphs are frequent

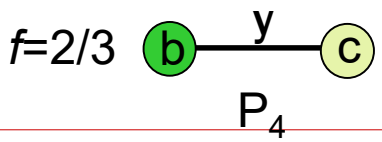
Other criteria for selecting subgraphs may be incorporated

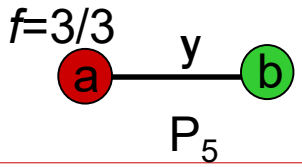


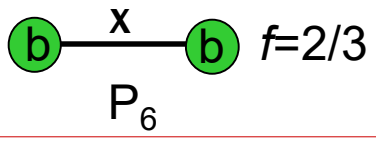




!







!: Maximal frequent subgraphs

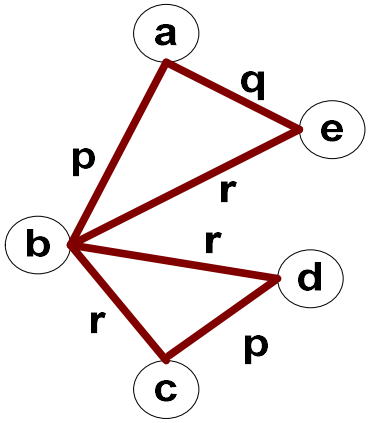
Graph Mining (Discovery of Frequent Substructures)

- Step 1: Generate frequent sub-structure candidates
- Step 2: Check for frequency of each candidate
  - Involves sub-graph isomorphism test which is computationally expensive
- Graph theory-based approaches
  - Apriori-based approach
  - Pattern-growth approach

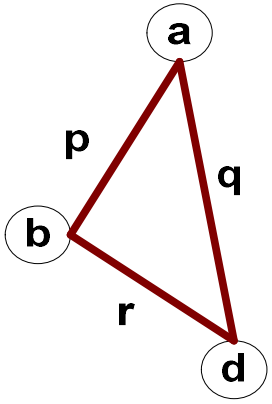
Apriori based Approach

- find **frequent** 1-subgraphs
- repeat
  - candidate generation
    - use frequent (k-1)-subgraph. to generate candidate k-sub.
  - candidate pruning
    - prune candidate subgraphs with **infrequent** (k-1)-subgraph.
  - support counting
    - count the support s for each remaining candidate
  - eliminate **infrequent** candidate k-subgraph.

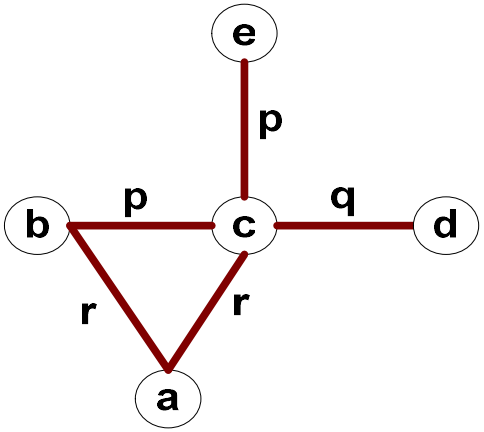
Example: Dataset



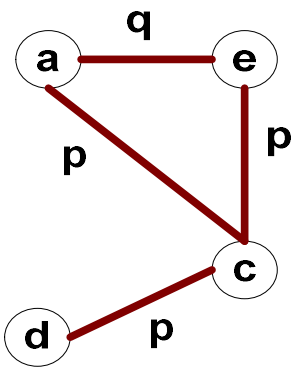
G1



G2



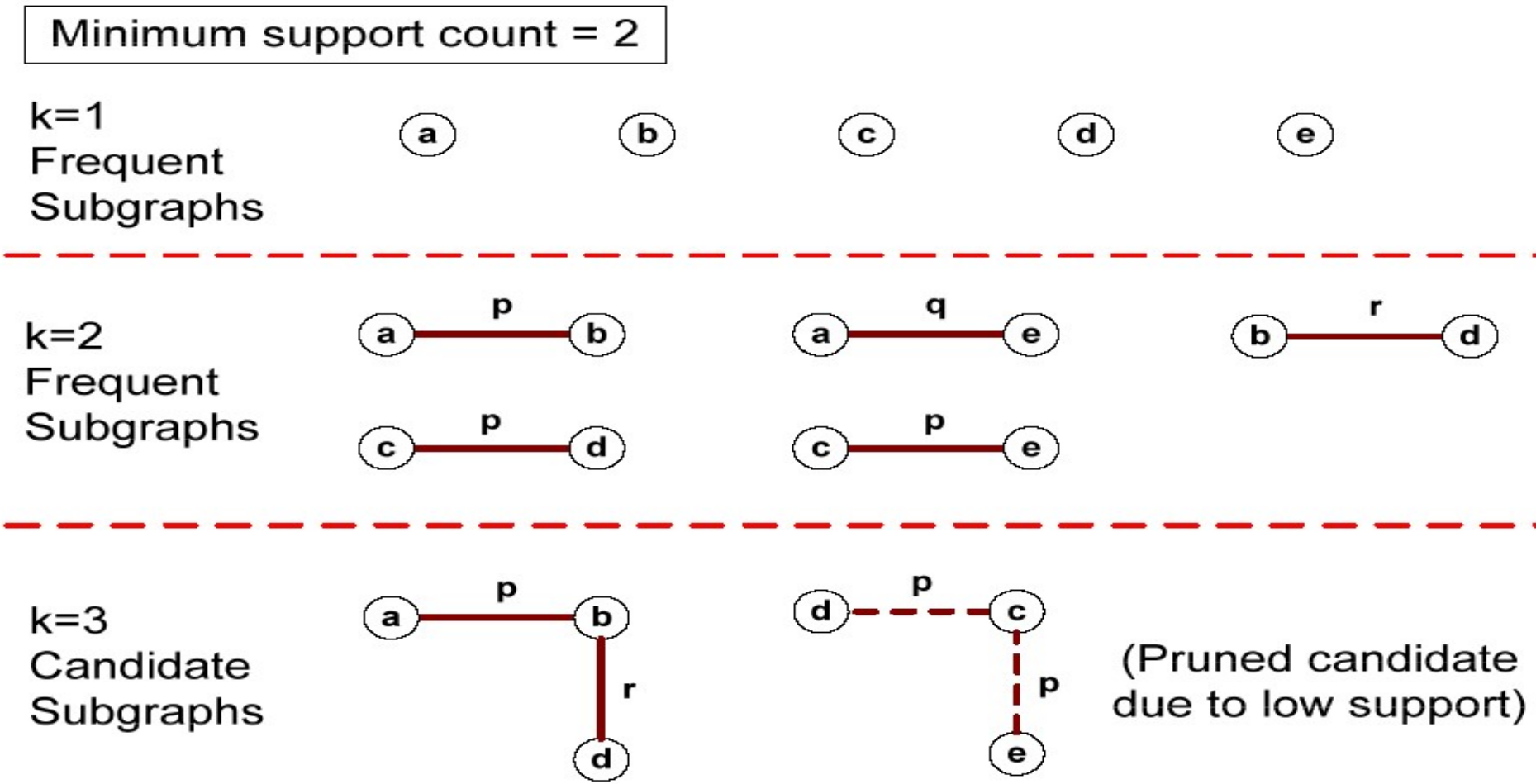
G3



G4

	(a,b,p)	(a,b,q)	(a,b,r)	(b,c,p)	(b,c,q)	(b,c,r)	…	(d,e,r)
G1	1	0	0	0	0	1	…	0
G2	1	0	0	0	0	0	…	0
G3	0	0	1	1	0	0	…	0
G4	0	0	0	0	0	0	…	0

a simple example



Candidate Generation

- In Apriori:
  - Merging two frequent k-itemsets will produce a candidate ($k+1$)-itemset

- In frequent subgraph mining
  (vertex/edge growing)
  - Merging two frequent k-subgraphs may produce more than one candidate ($k+1$)-subgraph

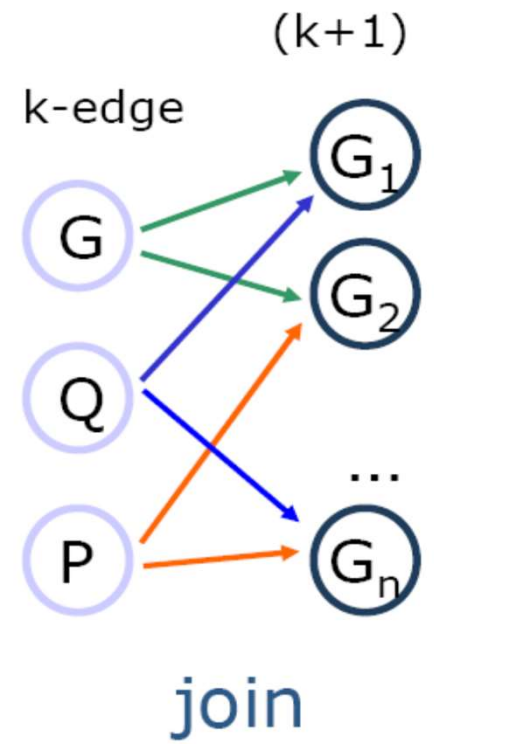
Apriori-based method

- Apriori Property
  - If a graph is frequent, all of its subgraphs are frequent.
- Candidate Generation
  - Create a set of candidate size k+1

    -from given two frequent k-subgraphs

    -containing the same (k-1)-subgraph

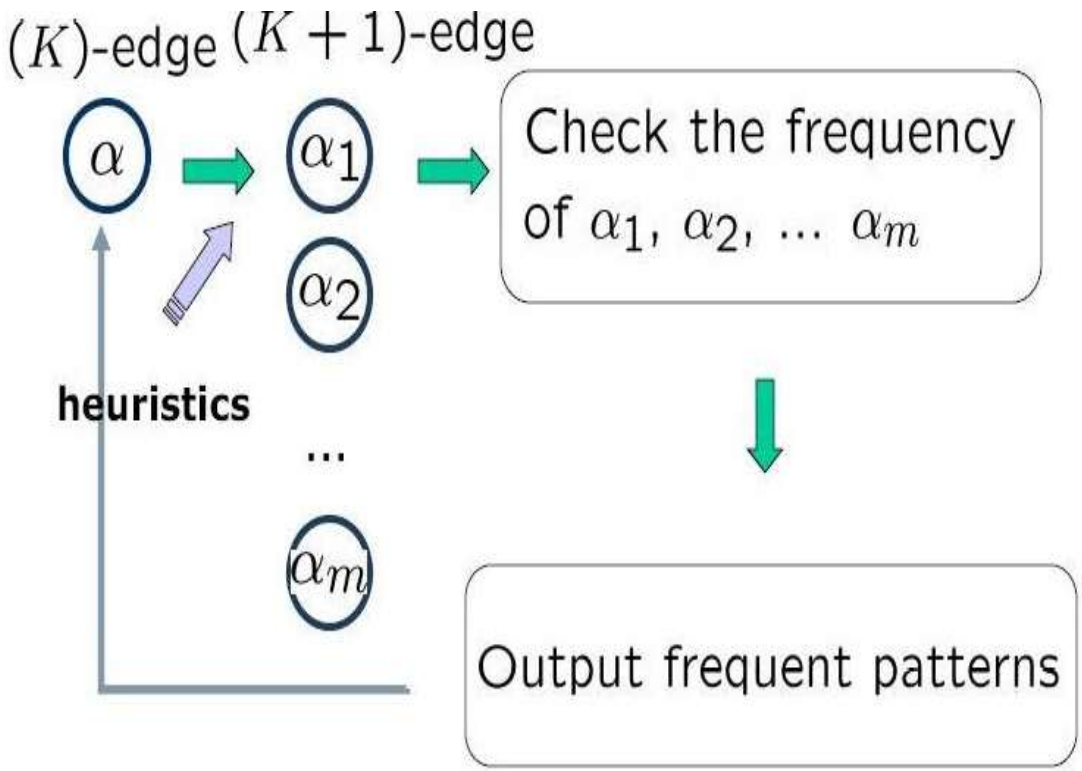
    -Result in several candidates size k+1



Apriori-Based Approach

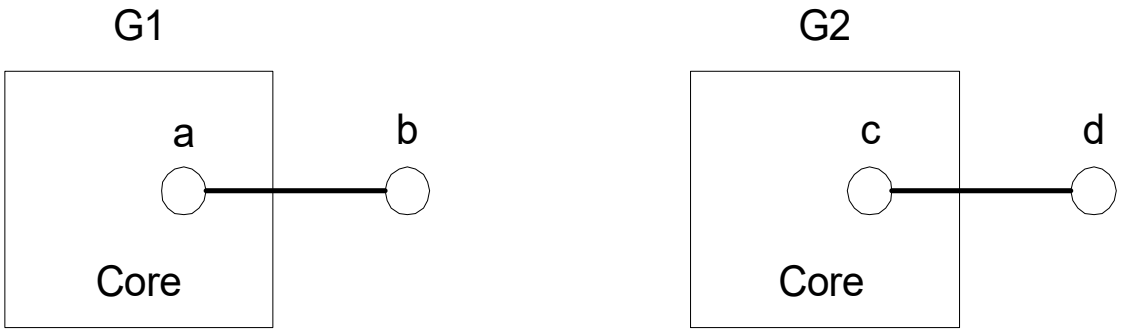
Apriori-based method

FlowChart

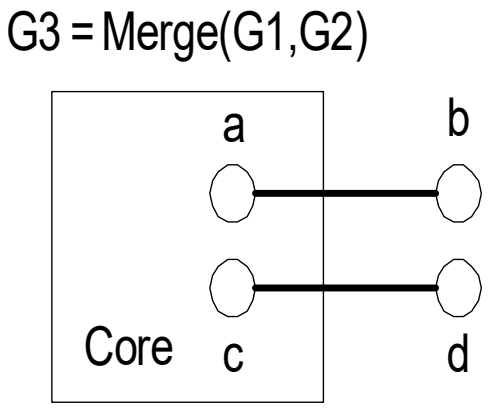


Candidate Generation by Edge Growing

- Given:

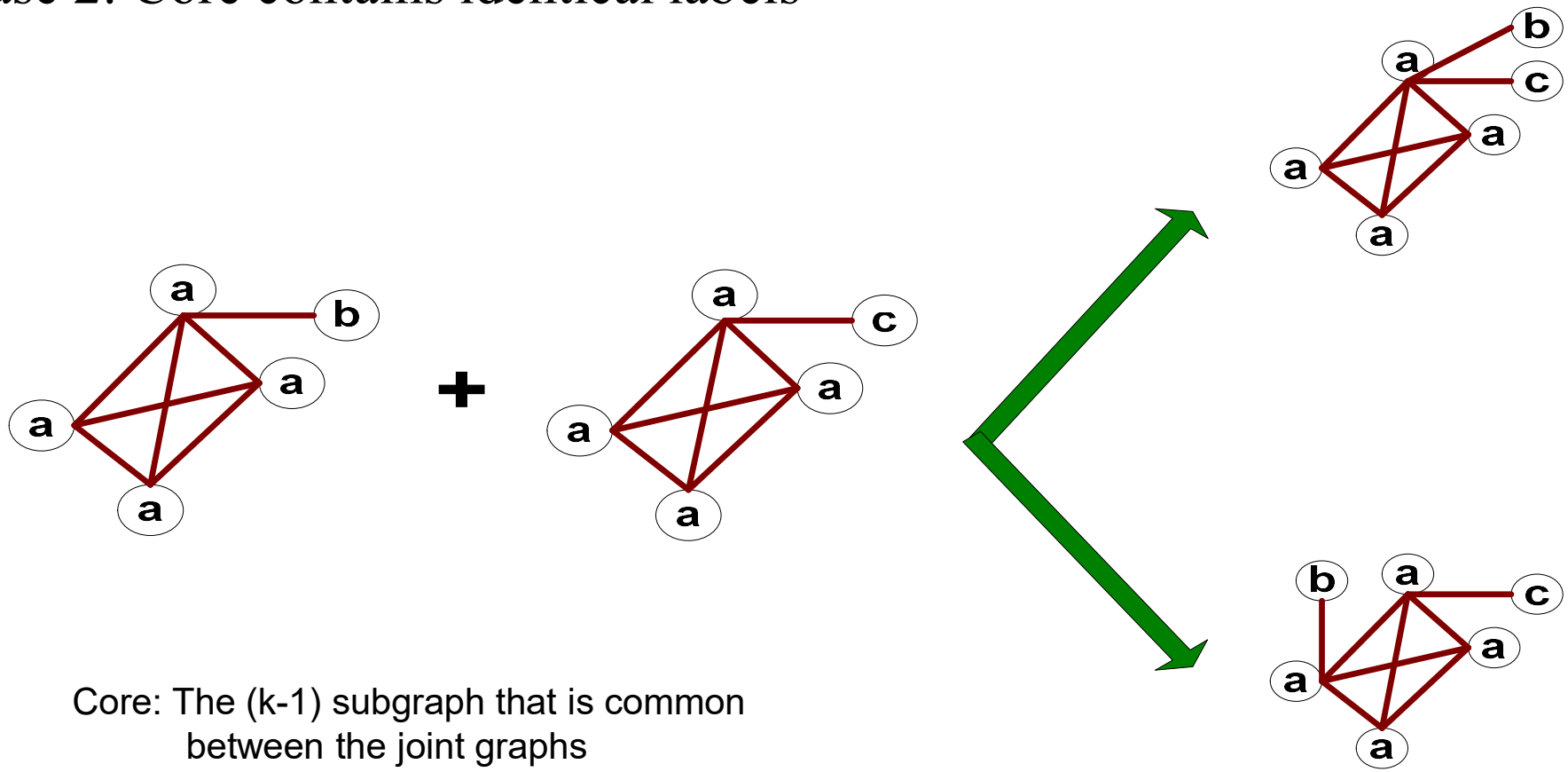


- Case 1: a ≠ c and b ≠ d



Candidate Generation by Edge Growing

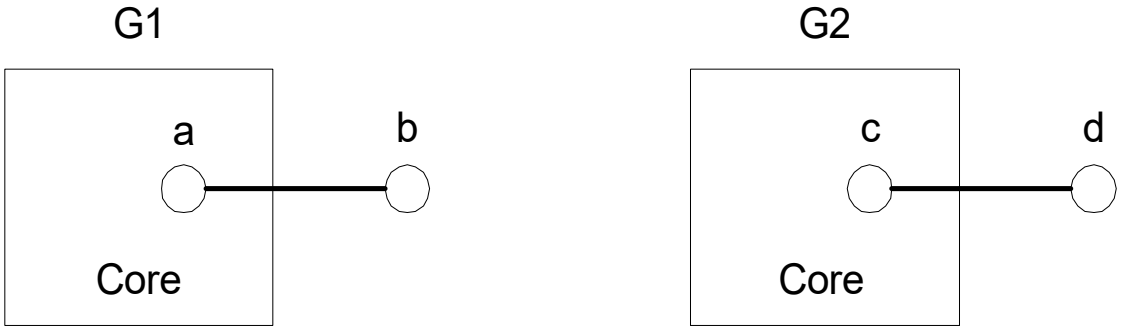
- Case 2: Core contains identical labels



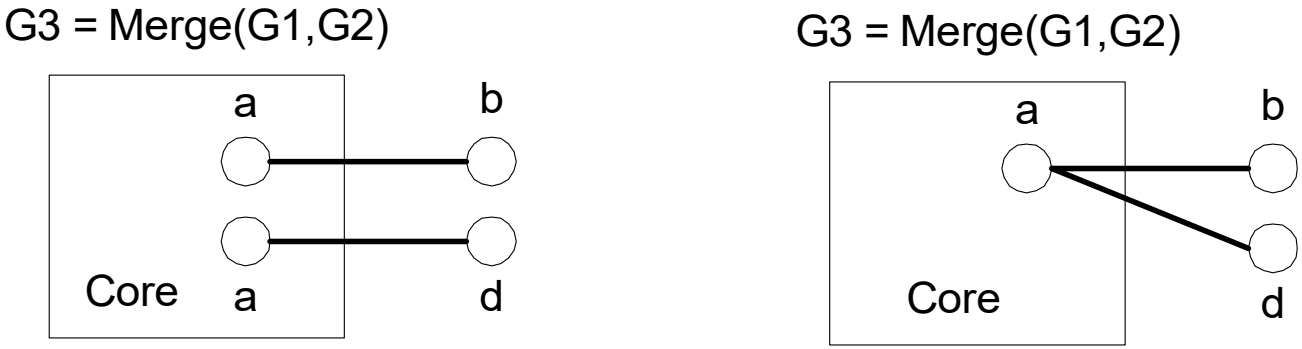
Core: The (k-1) subgraph that is common between the joint graphs

Candidate Generation by Edge Growing

- Given:

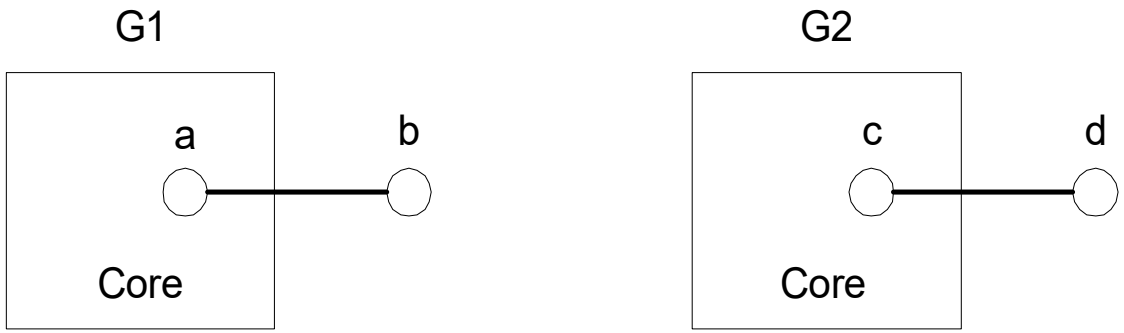


- Case 2: a = c and b ≠ d

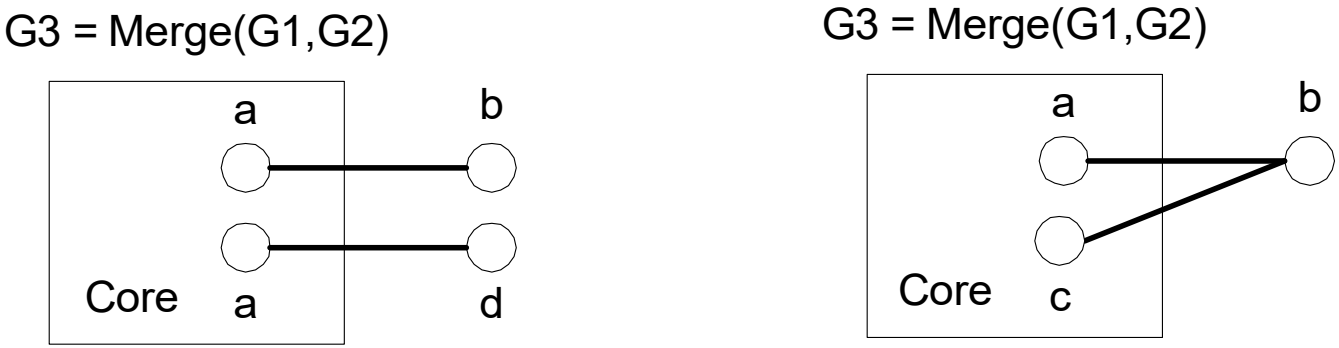


Candidate Generation by Edge Growing

- Given:

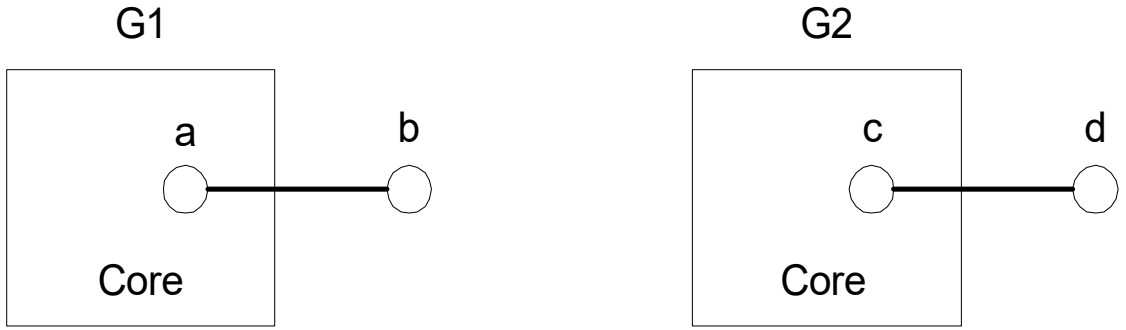


- Case 3: a ≠ c and b = d

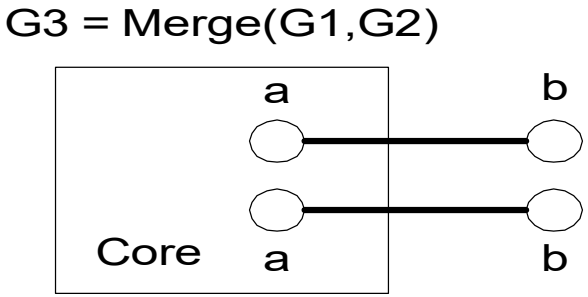


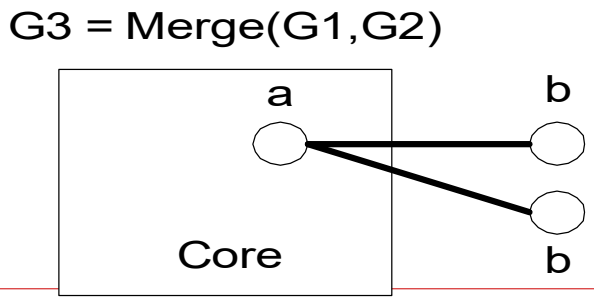
Candidate Generation by Edge Growing

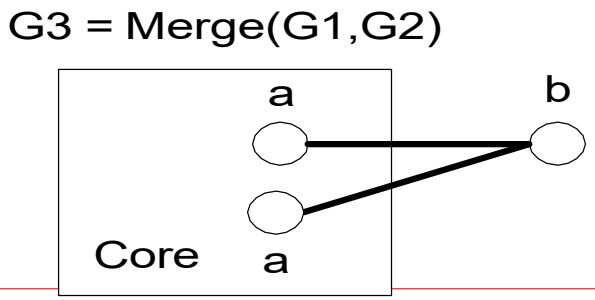
- Given:



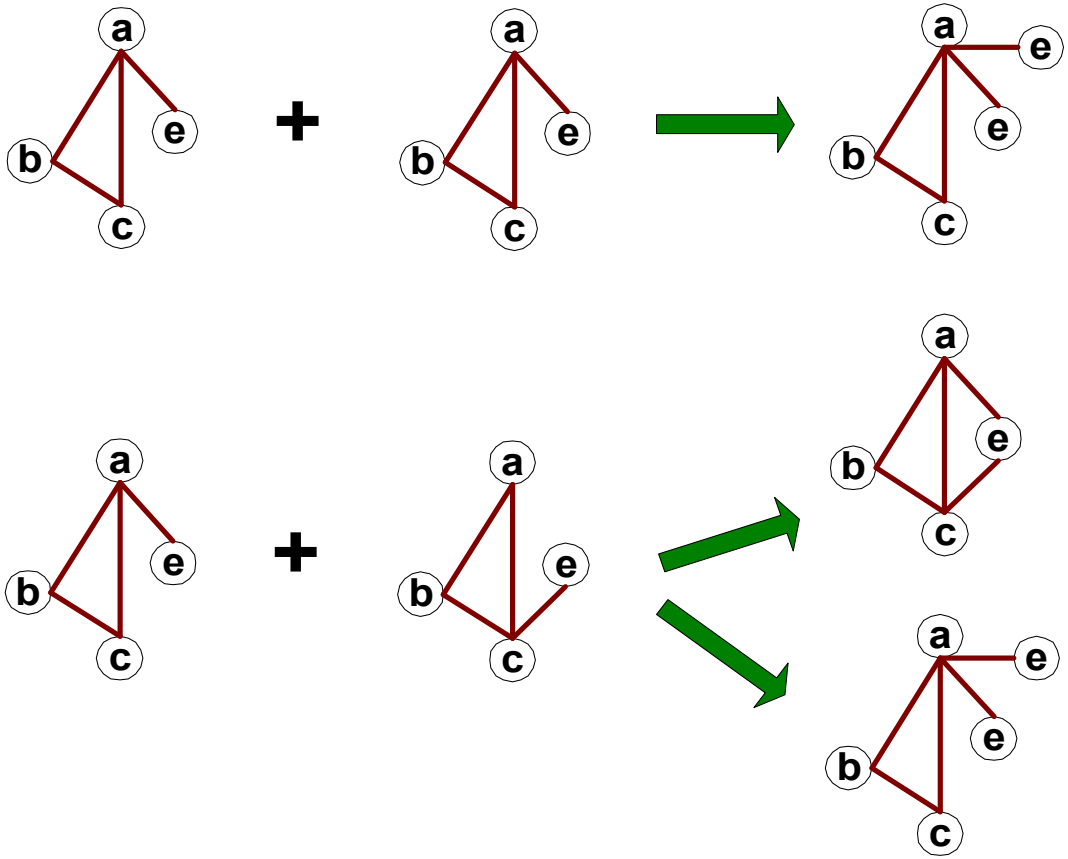
- Case 4: a = c and b = d





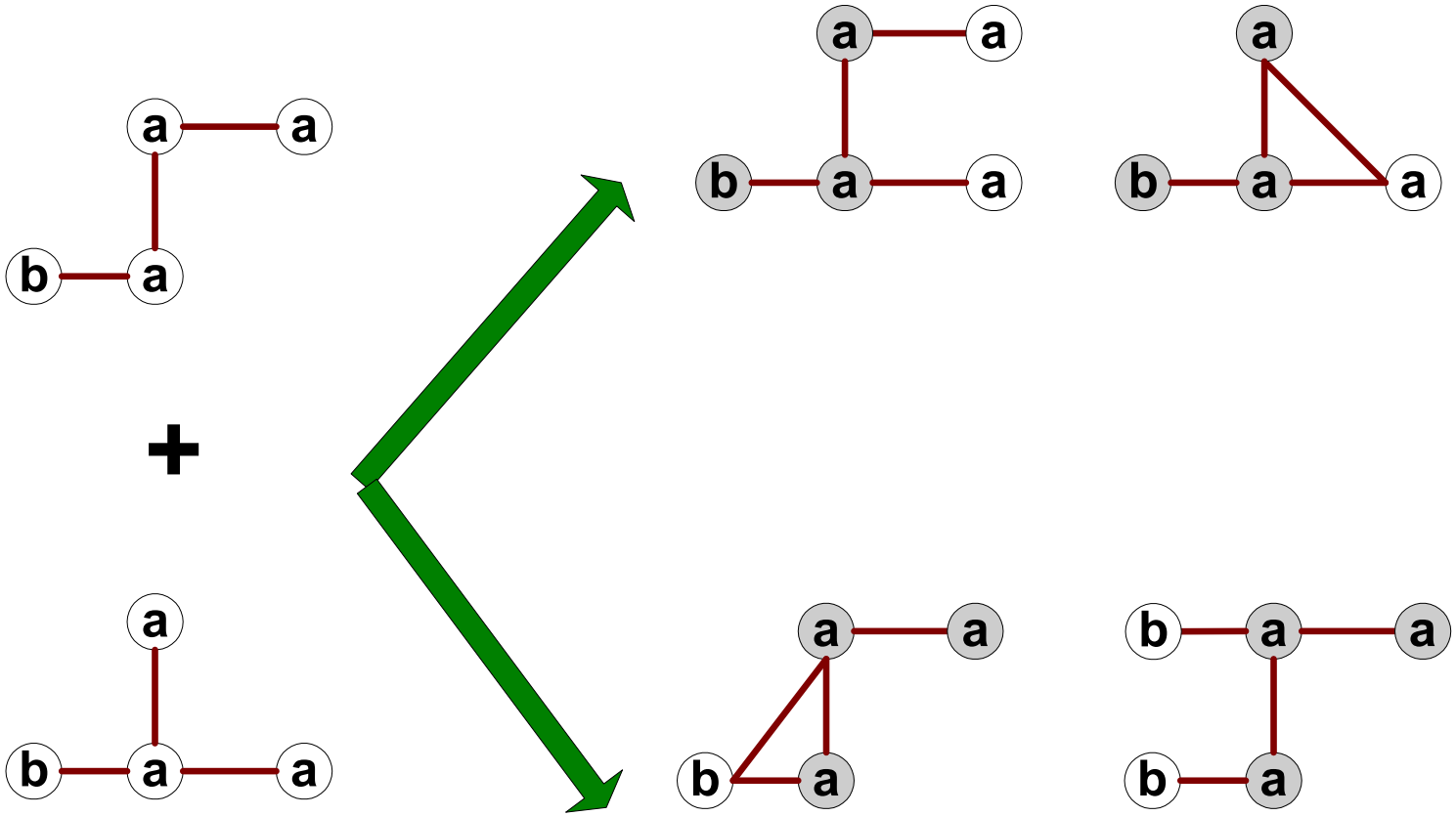


Candidate Generation by Edge Growing



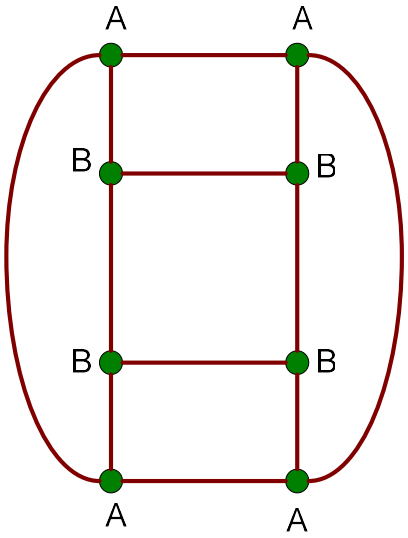
Candidate Generation by Edge Growing

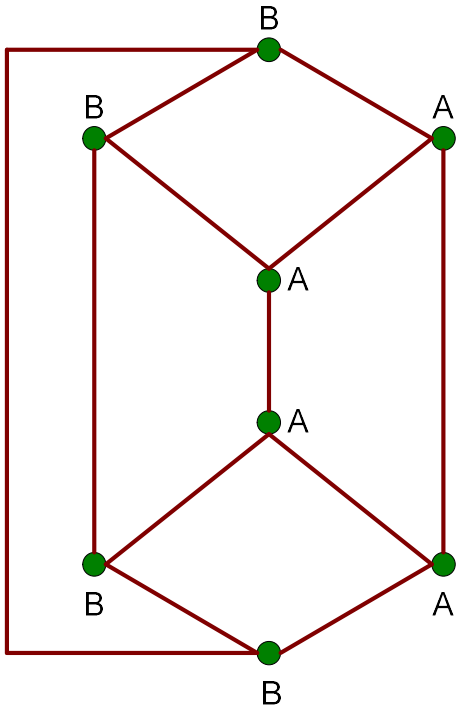
Core multiplicity



Graph Isomorphism

- A graph is isomorphic if it is topologically equivalent to another graph

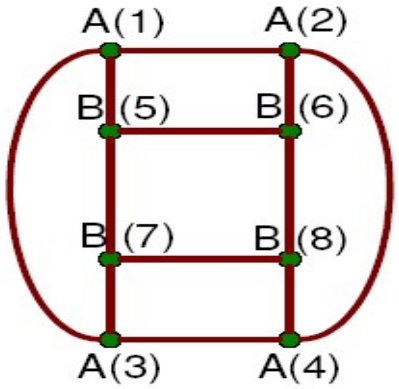




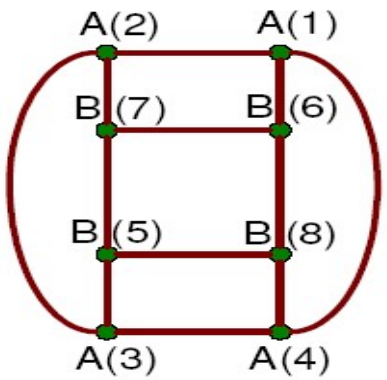
Graph Isomorphism

- Test for graph isomorphism is needed:
  - During candidate generation step, to determine whether a candidate has been generated
  - During candidate pruning step, to check whether its (k-1)-subgraphs are frequent
  - During candidate counting, to check whether a candidate is contained within another graph

graph representation: adjacency matrix



	A(1)	A(2)	A(3)	A(4)	B(5)	B(6)	B(7)	B(8)
A(1)	1	1	1	0	1	0	0	0
A(2)	1	1	0	1	0	1	0	0
A(3)	1	0	1	1	0	0	1	0
A(4)	0	1	1	1	0	0	0	1
B(5)	1	0	0	0	1	1	1	0
B(6)	0	1	0	0	1	1	0	1
B(7)	0	0	1	0	1	0	1	1
B(8)	0	0	0	1	0	1	1	1



	A(1)	A(2)	A(3)	A(4)	B(5)	B(6)	B(7)	B(8)
A(1)	1	1	0	1	0	1	0	0
A(2)	1	1	1	0	0	0	1	0
A(3)	0	1	1	1	1	0	0	0
A(4)	1	0	1	1	0	0	0	1
B(5)	0	0	1	0	1	0	1	1
B(6)	1	0	0	0	0	1	1	1
B(7)	0	1	0	0	1	1	1	0
B(8)	0	0	0	1	1	1	0	1

- The same graph can be represented in many ways

Apriori based Approach

Algorithm: AprioriGraph. Apriori-based frequent substructure mining.

Input:

- D, a graph data set;
- min_sup, the minimum support threshold.

Output:

- S_k, the frequent substructure set.

Method:

$S_1 \leftarrow$ frequent single-elements in the data set;
Call AprioriGraph(D, min_sup, S_1);

procedure AprioriGraph(D, min_sup, S_k)

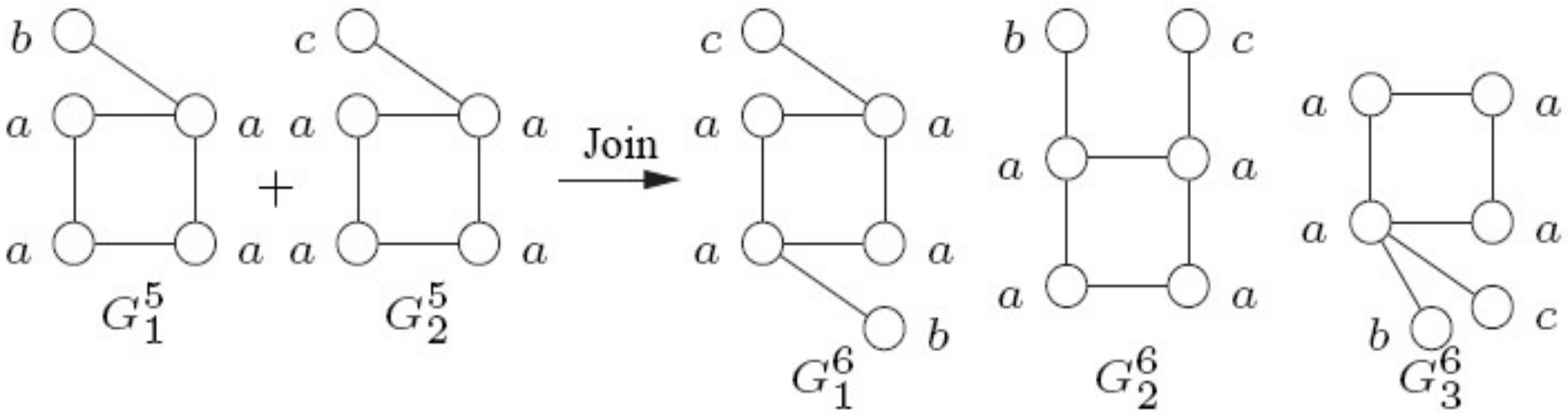
(1) $S_{k+1} \leftarrow \varnothing$;
(2) **for each** frequent $g_i \in S_k$ **do**
(3) **for each** frequent $g_j \in S_k$ **do**
(4) **for each** size $(k+1)$ graph g formed by the merge of g_i and g_j **do**
(5) **if** g is frequent in D and $g \notin S_{k+1}$ **then**
(6) insert g into S_{k+1};
(7) **if** $s_{k+1} \neq \varnothing$ **then**
(8) *AprioriGraph*(D, min_sup, S_{k+1});
(9) **return**;

Apriori based Approach

- Apriori-based approach
  - AGM/AcGM: Inokuchi, et al. (PKDD'00)
  - FSG: Kuramochi and Karypis (ICDM'01)
  - PATH#: Vanetik and Gudes (ICDM'02, ICDM'04)
  - FFSM: Huan, et al. (ICDM'03)

FSG algorithm: Frequent Sub-graph mining

- Edge-based Candidate generation – increases by one-edge at a time
- Two size k patterns are merged iff they share the same subgraph having k-1 edges (core)
- New candidate – has core and the two additional edges



FSG: Basic Flow of the Algo.

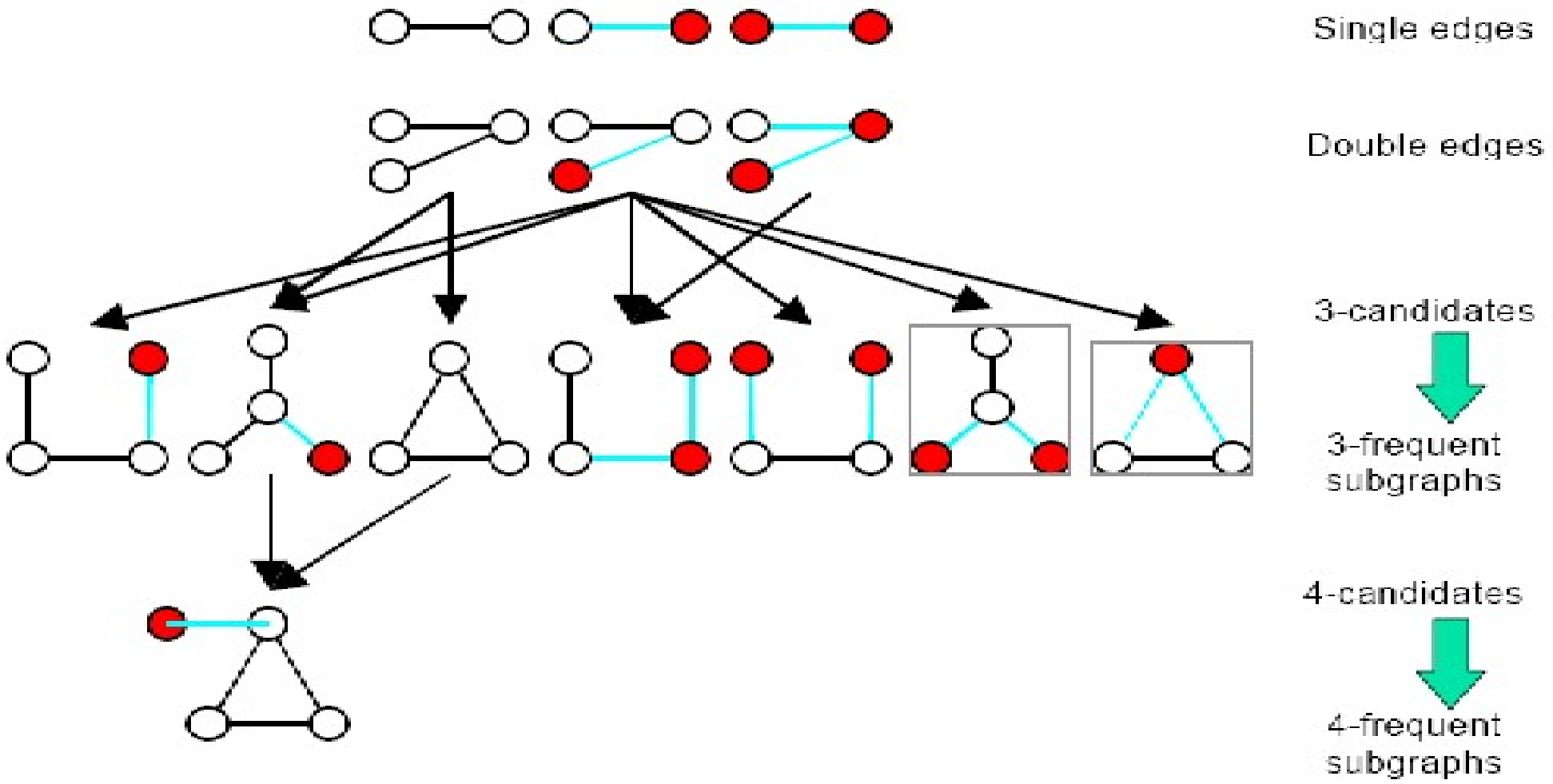
- Enumerate all single and double-edge subgraphs
- Repeat
  - Generate all candidate subgraphs of size (k+1) from size-k subgraphs
  - Prune candidate subgraphs with **infrequent** (k-1)-subgrapks by downward closure property
  - Count frequency of each remaining candidate by checking if a subgraph is contained in a transaction
  - Eliminate **infrequent** candidate k-subgraphs which don't satisfy support constraint

  Until (no frequent subgraphs at (k+1))

Join: Key Operation

- Join(L) = $\cup$ join(P, Q) for all P, Q $\in$ L
- Join(P, Q) = {G | P, Q, $\subset$ G, |G| = |P| + 1, |P| = |Q|}
- Two graphs P and Q are ***joinable*** if the join of the two graphs produces an non-empty set
- Theorem: two graphs P and Q are joinable if P $\cap$ Q is a graph with size |P| -1 or share a common “core” with size P-1

FSG Algorithm



Trivial operations with graphs…

- Candidate generation:
  - to determine two candidates for joining, we need to perform subgraph isomorphism for redundancy check
- Candidate pruning:
  - to check downward closure property, we need subgraph isomorphism again
- Frequency counting
  - subgraph isomorphism once again needed for checking containment of a frequent subgraphs
- Computational efficiency issue
  - how to reduce the number of graph/subgraph isomorphism operations?

FSG approach to frequency counting

Transactions

T1 = {f1, f2, f3}

T2 = {f1}

T3 = {f2}

Frequent Subgraphs

TID(f1) = {T1, T2}

TID(f2) = {T1, T3}

Candidate

c = join(f1, f2)

TID(c) = subset(TID(f1) AND TID(f2))

- Perform only subgraph_isomorph(c, T1)
- TID lists trade memory for performance.

frequency counting algorithmically

frequency counting

keep track of the TID lists

if a size k-candidate is contained in a transaction, all the size (k – 1)-parents must be contained in the same transaction

perform subgraph isomorphism only on the intersection of the TID lists of the parent frequent subgraphs of size k – 1

remarks:

- significantly reduces the number of subgraph
- isomorphism checks; trade-off between running time and memory

Apriori based Approach

- Edge disjoint path method
  - Classify graphs by number of disjoint paths they have
  - Two paths are edge-disjoint if they do not share any common edge
  - A substructure pattern with k+1 disjoint paths is generated by joining sub-structures with k disjoint paths
- Disadvantage of Apriori Approaches
  - Overhead when joining two sub-structures
  - Uses BFS strategy : level-wise candidate generation
  - To check whether a k+1 graph is frequent – it must check all of its size-k sub graphs
  - May consume more memory

Pattern Growth Approach

- Weakness of Apriori-based approach
  - The generation of size (k+1) subgraph candidates from size k frequent subgraph too complicated and complex.
  - Pruning false positive : subgraph isomorphism is an NP complete problem which is costly.
- Uses BFS as well as DFS
  - A graph g can be extended by adding a new edge e. The newly formed graph is denoted by g ♦x e.
  - Edge e may or may not introduce a new vertex to g.
  - If e introduces a new vertex, the new graph is denoted by g ♦xf e, otherwise, g ♦xb e, where f or b indicates that the extension is in a forward or backward direction.

Pattern Growth Approach

- Pattern Growth Approach
  - For each discovered graph g performs extensions recursively until all frequent graphs with g are found
  - Simple but inefficient
  - Same graph is discovered multiple times – duplicate graph
- Methods:
  - MoFa, Borgelt and Berthold (ICDM'02)
  - gSpan: Yan and Han (ICDM'02)
  - Gaston: Nijssen and Kok (KDD'04)

gSpan: Graph-Based Substructure Pattern Mining

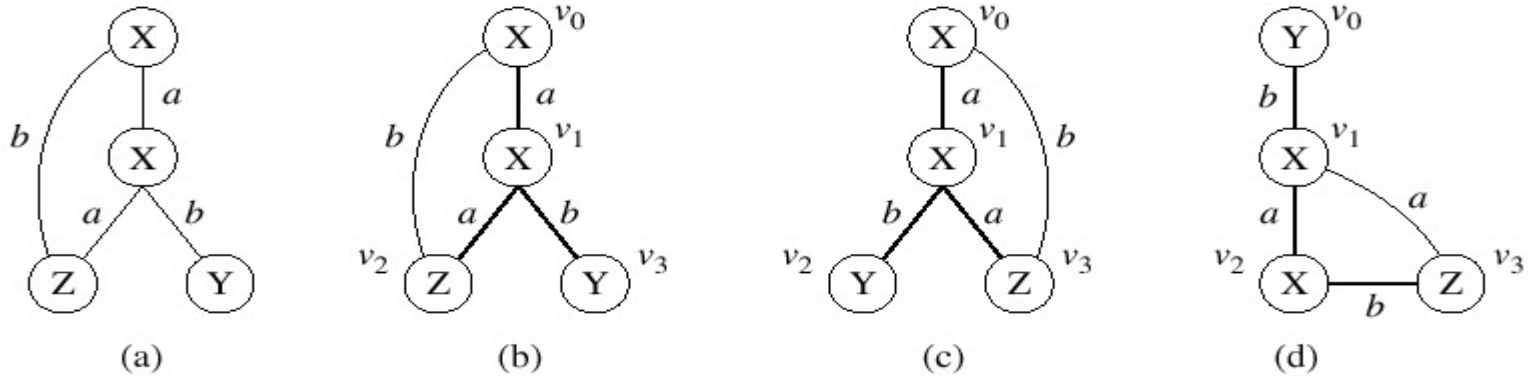
- gSpan: Graph-Based Substructure Pattern Mining
  - Change the way to represent a graph (DFS: Depth First Search)
  - Using pattern growth to generate new subgraph candidate.

- DFS (Depth First Search) Code
  - First Step: DFS the graph and use edges on the path to represent the graph.
  - Second Step: DFS Lexicographic Order

- Pattern Growth subgraph generation

gSpan Algorithm

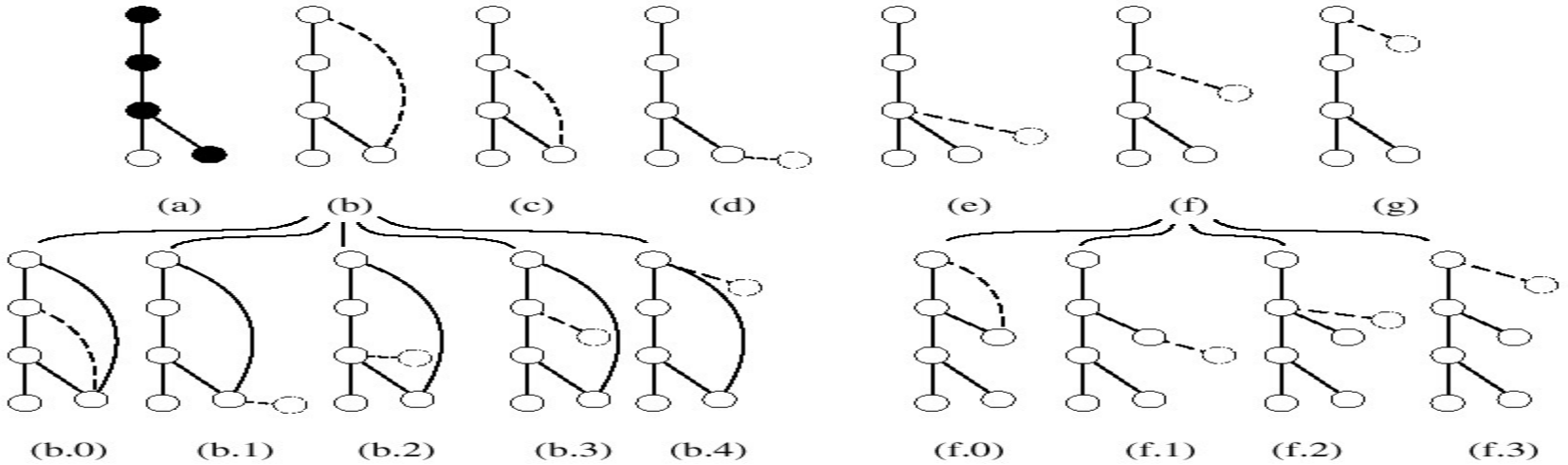
- Reduces generation of duplicate graphs
  - Does not extend duplicate graphs
  - Uses Depth First Order
  - A graph may have several DFS-trees
    - Visiting order of vertices forms a linear order – Subscript
    - In a DFS tree – starting vertex – root; last visited vertex – right-most vertex
    - Path from v_0 to v_n – right most path



Right most path: (b), (c) – (v_0, v_1, v_3); (d) – (v_0, v_1, v_2, v_3)

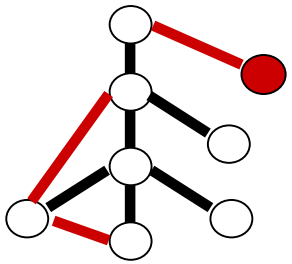
gSpan Algorithm

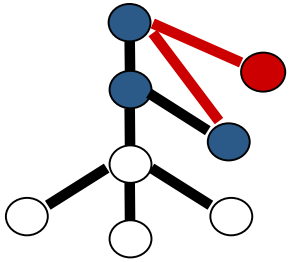
- gSpan restricts the extension method
  - A new edge e can be added
    - between the right-most vertex and another vertex on the right-most path (backward extension);
    - or it can introduce a new vertex and connect to a vertex on the right-most path (forward extension)
  - Right-most extension, denoted by G ♦r e



gSpan (Yan and Han ICDM'02)

Right-Most Extension





Theorem: Completeness

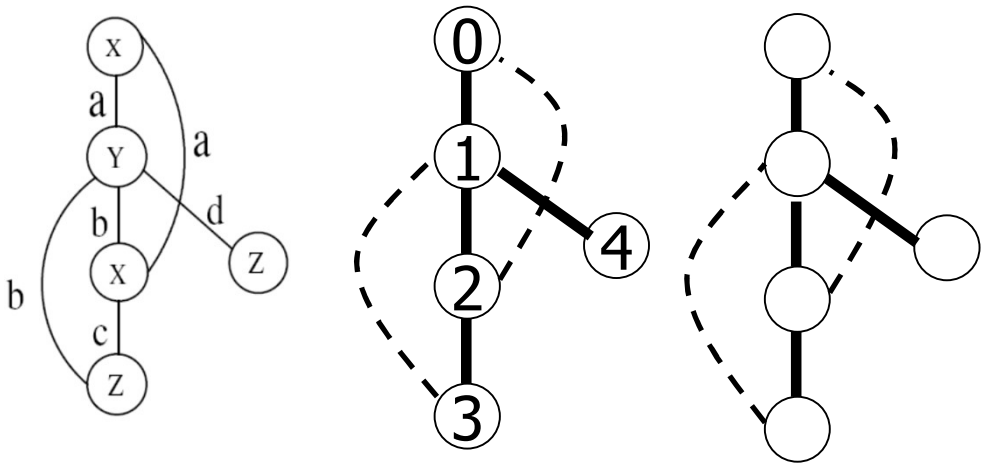
The Enumeration of Graphs using Right-most Extension is COMPLETE

gSpan Algorithm

- Chooses any one DFS tree – base subscripting and extends it
  - Each subscripted graph is transformed into an edge sequence – DFS code
  - Select the subscript that generates minimum sequence
    - Edge Order – maps edges in a subscripted graph into a sequence
    - Sequence Order – builds an order among edge sequences

DFS Code

- Flatten a graph into a sequence using depth first search



e0: (0,1)

e1: (1,2)

e2: (2,0)

e3: (2,3)

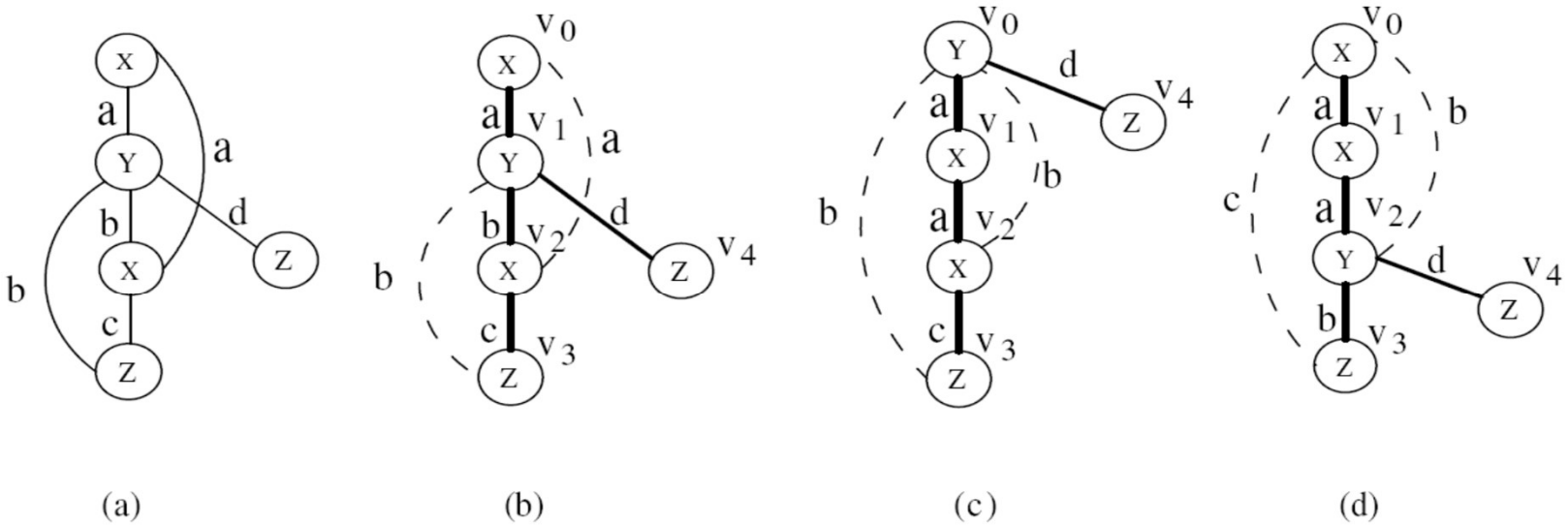
e4: (3,1)

e5: (1,4)

DFS code: (0, 1, x, a, y), (1, 2, y, b, x), (2, 0, x, a, x) , (2, 3, x, c, z),

(3, 1, z, b, y), (1, 4, y, d, z)

DFS Code

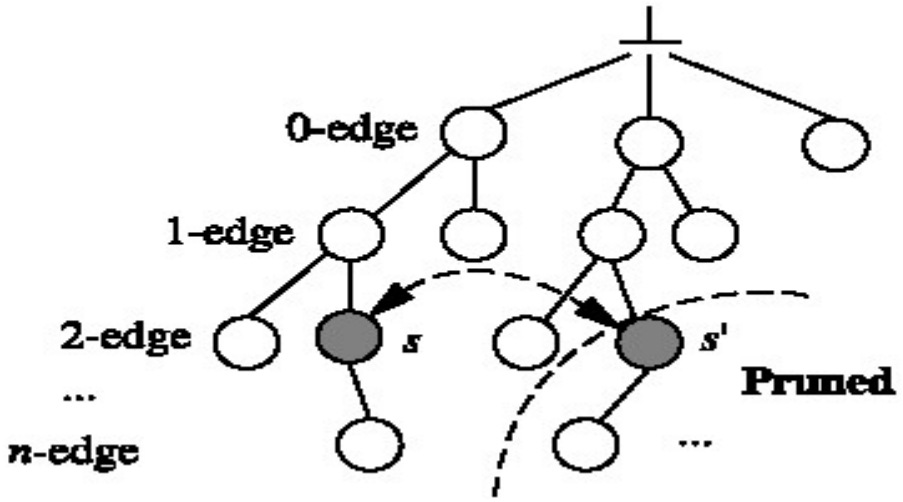


edge	(Fig 1b) α	(Fig 1c) β	(Fig 1d) γ
0	$(0, 1, X, a, Y)$	$(0, 1, Y, a, X)$	$(0, 1, X, a, X)$
1	$(1, 2, Y, b, X)$	$(1, 2, X, a, X)$	$(1, 2, X, a, Y)$
2	$(2, 0, X, a, X)$	$(2, 0, X, b, Y)$	$(2, 0, Y, b, X)$
3	$(2, 3, X, c, Z)$	$(2, 3, X, c, Z)$	$(2, 3, Y, b, Z)$
4	$(3, 1, Z, b, Y)$	$(3, 0, Z, b, Y)$	$(3, 0, Z, c, X)$
5	$(1, 4, Y, d, Z)$	$(0, 4, Y, d, Z)$	$(2, 4, Y, d, Z)$

Table 1. DFS codes for Fig. 1(b)-(d)

gSpan Algorithm

- Root – Empty code
  - Each node is a DFS code encoding a graph
  - Each edge – rightmost extension from a (k-1) length DFS code to a k-length DFS code
    - If codes s and s' encode the same graph – search space s' can be safely pruned



DFS Code Extension

- Let ***a*** be the minimum DFS code of a graph G and ***b*** be a non-minimum DFS code of G. For any graph G' $\supset$ G, we have
  - minDFS(G') < b
  - a is a prefix of minDFS(G') or minDFS(G') < a
- There is a 1-1 mapping from a graph to its minimum DFS code.
- For every graph G', there exists a G such that G' $\supset$ G and minDFS(G) is a prefix of minDFS(G')

gSpan Code

- Input: A graph database and a support threshold t
- Output: All frequent patterns F

gSpan:

F_1 = { frequent node labels}, K=1,

gSpan_enumeration (F_1, K, F)

gSpan_enumeration (F_K, K, F)

K = K + 1;

For each pattern P in F_k

C = Candidates(P, K);

F = $F \cup C$;

gSpan_enumeration (C, K, F)

Graph Pattern Explosion Problem

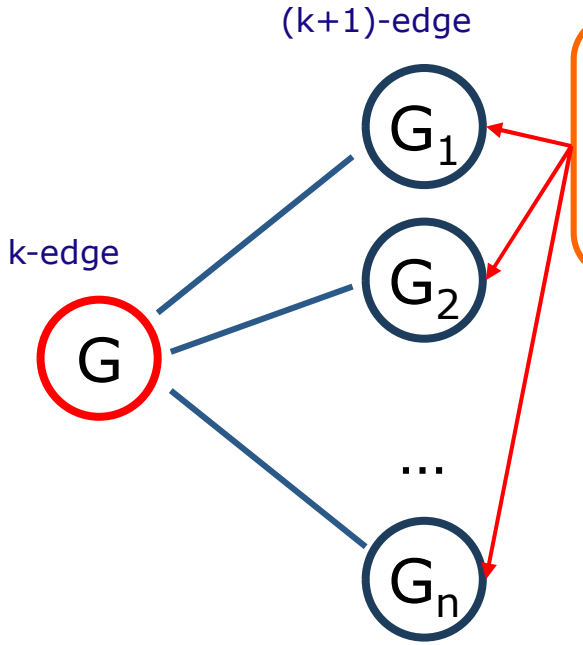
- If a graph is frequent, all of its subgraphs are frequent — **the Apriori property**
- An **n**-edge frequent graph may have 2^n subgraphs
- Among **422** chemical compounds which are confirmed to be active in an AIDS antiviral screen dataset, there are **1,000,000** frequent graph patterns if the minimum support is 5%

Mining Closed Frequent Graphs

- Motivation: Handling graph pattern explosion problem
- Closed frequent graph
  - A frequent graph G is *closed* if there exists no supergraph of G that carries the same support as G
- If some of G's subgraphs have the same support, it is unnecessary to output these subgraphs (nonclosed graphs)
- *Lossless compression:* still ensures that the mining result is complete
- A frequent pattern G is maximal if and only if there is no frequent superpattern of G.
- Maximal pattern set is a subset of the closed pattern set.
  - But cannot be used to reconstruct entire set of frequent patterns

CLOSEGRAPH (Yan & Han, KDD'03)

A Pattern-Growth Approach



At what condition, can we stop searching their children i.e., early termination?

If G and G' are frequent, G is a subgraph of G'. If in any part of the graph in the dataset where G occurs, G' also occurs, then we need not grow G, since none of G's children will be closed except those of G'.

Classification and Cluster Analysis using Graph Patterns

- Graph Classification
  - Mine frequent graph patterns
    - Features that are frequent in one class but less in another – Discriminative features – Model construction
    - Can adjust frequency, connectivity thresholds
    - SVM, NBM etc are used
- Cluster Analysis
  - Cluster Similar graphs based on graph connectivity (minimal cuts)
  - Hierarchical clusters based on support threshold
  - Outliers can also be detected
- Inter-related process

Graph Clustering

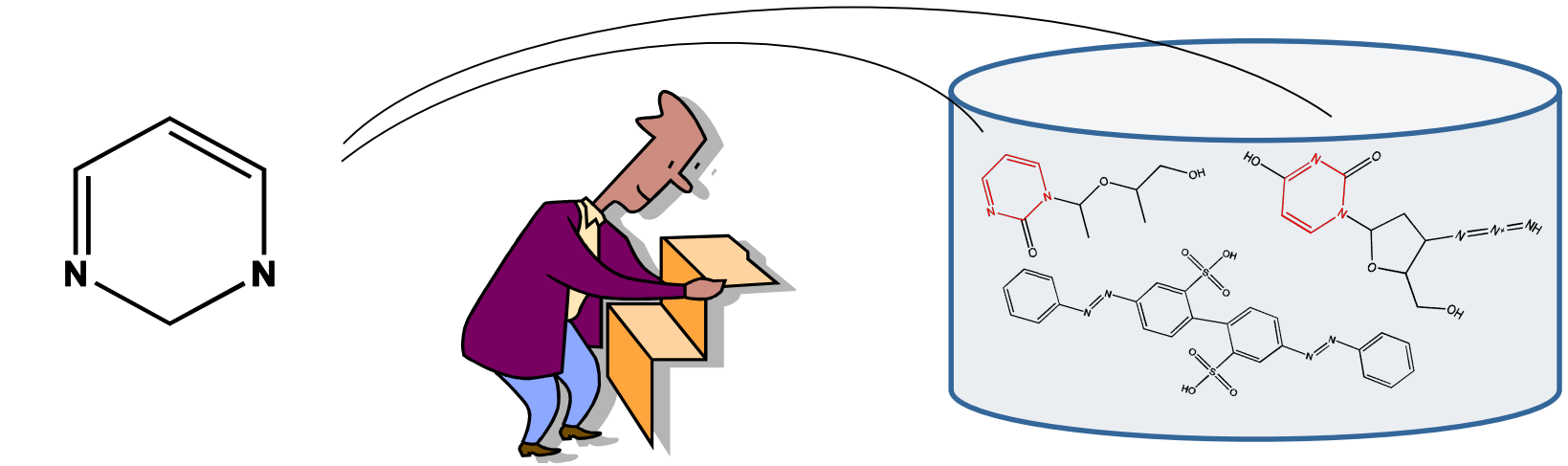
- Graph similarity measure
  - Feature-based similarity measure
    - Each graph is represented as a feature vector
    - The similarity is defined by the distance of their corresponding vectors
    - Frequent subgraphs can be used as features
  - Structure-based similarity measure
    - Maximal common subgraph
    - Graph edit distance: insertion, deletion, and relabel
    - Graph alignment distance

Graph Classification

- Local structure based approach
  - Local structures in a graph, e.g., neighbors surrounding a vertex, paths with fixed length
- Graph pattern-based approach
  - Subgraph patterns from domain knowledge
  - Subgraph patterns from data mining
- Kernel-based approach
  - Random walk (Gärtner ’02, Kashima et al. ’02, ICML’03, Mahé et al. ICML’04)
  - Optimal local assignment (Fröhlich et al. ICML’05)
- Boosting (Kudo et al. NIPS’04)

Graph Search

- Querying graph databases:
  - Given a graph database and a query graph, find all the graphs containing this query graph



Summary: Graph Mining

- Graph mining has wide applications
- Frequent and closed subgraph mining methods
  - gSpan and CloseGraph: pattern-growth depth-first search approach
- Similarity search in graph databases
  - Indexing and feature-based matching
- Further development and application exploration